

DSE/GE4b: Introduction to Web Programming

Study Notes — Units 3, 4 & 5

CSS | JavaScript | jQuery & JSON

UNIT 3: CASCADING STYLE SHEETS (CSS)

Ch. 4 (HTML 143–185) | Book [1]: *Responsive Web Design with HTML5 & CSS*

3.1 Introduction to CSS

CSS (Cascading Style Sheets) is a language used to describe the formatting/presentation of HTML documents. While HTML provides the structure and content of a webpage, CSS controls its appearance — colors, fonts, spacing, layout, and more. CSS allows web designers to separate content from style, making websites easier to redesign and maintain.

CSS History: CSS1 (1996): basic text, color, borders. CSS2 (1998): positioning. CSS3 (2001+): shadows, rounded corners, animations, media queries.

3.2 Types of Style Sheets & Precedence

CSS has three ways to apply styles. The word "cascading" refers to the priority order — styles closer to the content always take precedence (highest to lowest priority):

1. Inline Styles — Highest Priority

Applied using the style attribute inside any HTML tag. Applies to that ONE element only. Use sparingly — defeats the purpose of separating content from style.

```
<h1 style="color: navy; font-size: 28px;">Welcome!</h1>
<p style="background-color: yellow; padding: 10px;">Highlighted text</p>
```

2. Embedded (Internal) Style Sheets

Written inside <style> tags in the <head> section. Applies only to that single webpage. Takes precedence over external style sheets.

```
<head>
  <style>
    body { background-color: #f0f0f0; }
    h1   { color: #1a5276; font-family: Arial; }
  </style>
</head>
```

3. External Style Sheets — Recommended

A separate .css file linked to all HTML pages. Changing one file updates all linked pages (called "reskinning"). Best practice for most web projects.

```
/* styles.css */
```

```
body { font-family: Arial, sans-serif; margin: 0; padding: 0; }
h1   { color: #1a5276; }
```

Link it in the <head> of every HTML file:

```
<link rel="stylesheet" href="styles.css">
```

Cascade / Precedence Order: Inline > Embedded > External > Browser default

Inheritance: Child elements inherit styles from parent (e.g., <p> inherits body font).

Specificity: More specific selector wins (e.g., #id beats .class beats element).

3.3 CSS Syntax: Selectors & Declarations

```
selector {
  property: value;    /* one declaration */
  property: value;    /* another declaration */
}
```

Component	Description	Example
Selector	Identifies what to style	body, h1, #nav, .box
Property	The style attribute to change	color, font-size, margin
Value	The specific setting	blue, 16px, 2em, #ff0000
Declaration	One property:value pair	color: blue;
Rule-Set	Full selector + declaration block	h1 { color: blue; }

3.4 CSS Selectors

Selector	Syntax	Description	Example
Element	element	All elements of that type	p { color: black; }
ID	#id	One unique element (used once per page)	#header { background: navy; }
Class	.class	Multiple elements sharing a class	.highlight { color: red; }
Universal	*	ALL elements on the page	* { box-sizing: border-box; }
Group	a, b, c	Apply same style to multiple selectors	h1, h2, h3 { font-family: Arial; }
Descendant	a b	Elements b that are inside a	nav a { color: white; }
Child	a > b	Direct child b of a	ul > li { list-style: none; }
Pseudo-class	a:state	Element in a specific state	a:hover { color: red; }
Attribute	[attr]	Elements with a specific attribute	input[type="text"] { }

Common Pseudo-classes

Pseudo-class	Description
a:link	Unvisited hyperlink
a:visited	Already visited hyperlink
a:hover	Mouse is hovering over element
a:active	Element is being clicked
input:focus	Input element has keyboard focus
:nth-child(even)	Every even child (great for table striping)
:first-child	First child element
:last-child	Last child element

3.5 CSS Text & Font Properties

Property	Description	Common Values
font-family	Typeface/font name	Arial, Verdana, "Times New Roman", serif, sans-serif
font-size	Text size	16px, 1.2em, 120%, small, large
font-weight	Boldness	normal, bold, bolder, lighter, 400, 700
font-style	Italic or normal	normal, italic, oblique
color	Text color	blue, #1a5276, rgb(255,0,0), rgba(0,0,0,0.5)
text-align	Horizontal alignment	left, right, center, justify
text-decoration	Underline, strikethrough	none, underline, line-through, overline
text-transform	Change letter case	uppercase, lowercase, capitalize
line-height	Space between lines	1.5, 24px, 150%
letter-spacing	Space between letters	2px, 0.1em
word-spacing	Space between words	5px, 0.5em

3.6 CSS Colors

CSS supports multiple color formats:

```
color: red; /* Named color */
color: #ff0000; /* Hexadecimal (RRGGBB) */
color: #f00; /* Hex shorthand (RGB) */
color: rgb(255, 0, 0); /* RGB (0-255 per channel) */
color: rgba(255, 0, 0, 0.5); /* RGB + Alpha (0=transparent, 1=opaque) */
color: hsl(0, 100%, 50%); /* Hue(0-360) Saturation% Lightness% */
```

3.7 CSS Box Model

Every HTML element is treated as a rectangular box with 4 layers (inside → out):

Layer	CSS Property	Description
Content	width, height	The actual text/image content area

Padding	padding	Space inside the border, between content and border. Part of the element's background.
Border	border	Visible line/boundary around the element
Margin	margin	Space OUTSIDE the border, between elements. Always transparent.

```
/* Shorthand: Top Right Bottom Left (clockwise) */
margin: 10px 20px 15px 5px;
padding: 10px 20px; /* top/bottom=10, left/right=20 */
border: 2px solid #333; /* width style color */
box-sizing: border-box; /* width includes padding+border (recommended) */
```

3.8 Inline vs Block Elements

Block Elements	Inline Elements
Take full width available	Take only as much width as their content
Always start on a new line	Stay on same line as surrounding content
width/height can be set in CSS	width/height ignored by CSS
Examples: <div>, <p>, <h1>-<h6>, , , <table>, <form>	Examples: , <a>, , , , <input>

You can change an element's display behavior:

```
display: block; /* make inline element behave as block */
display: inline; /* make block element behave as inline */
display: inline-block; /* inline but allows width/height */
display: none; /* hide element completely */
```

3.9 CSS for Structural/Semantic Elements

HTML5 semantic elements (header, nav, main, aside, footer) can be targeted directly as CSS selectors:

```
header { background-color: #1a5276; color: white; padding: 20px; }
nav { background-color: #2e86c1; }
nav a { color: white; text-decoration: none; padding: 10px; display: inline-block; }
nav a:hover { background-color: #1a5276; }
main { max-width: 960px; margin: 0 auto; padding: 20px; }
aside { float: right; width: 30%; background: #f0f0f0; padding: 15px; }
footer { background: #333; color: #aaa; text-align: center; padding: 15px; }
```

3.10 ID Selectors & Class Attributes

ID Selectors (#)

IDs must be UNIQUE on a page — use for one specific element only. In CSS, prefix with #.

```
<div id="main-header">Site Header</div>

/* CSS */
```

```
#main-header { background: navy; color: white; padding: 20px; }
```

Class Attributes (.)

Classes can be applied to MANY elements. In CSS, prefix with a dot (.). One element can have multiple classes.

```
<p class="highlight important">This is important!</p>
<button class="btn btn-primary">Click Me</button>

/* CSS */
.highlight { background-color: yellow; }
.important { font-weight: bold; color: red; }
.btn { padding: 10px 20px; border-radius: 5px; cursor: pointer; }
.btn-primary { background-color: #1a5276; color: white; }
```

3.11 CSS Webpage Layout

Float Layout (Traditional)

```
.left-column { float: left; width: 30%; }
.right-column { float: left; width: 70%; }
.footer      { clear: both; } /* clear stops float effect */
```

Flexbox Layout (Modern)

```
.container { display: flex; justify-content: space-between; align-items: center; }
.item      { flex: 1; }
```

CSS Grid (Modern)

```
.grid { display: grid; grid-template-columns: 1fr 2fr 1fr; gap: 20px; }
```

3.12 Common Background Properties

Property	Description	Example
background-color	Solid background color	background-color: #f0f0f0;
background-image	Image as background	background-image: url("bg.jpg");
background-repeat	Repeat behavior	no-repeat, repeat-x, repeat-y
background-position	Position of background	center top, left bottom
background-size	Size of background image	cover, contain, 100% auto

3.13 Adding Comments & Validating CSS

CSS Comments: use `/* comment text */` (NOT `// style`). Comments are ignored by the browser.

```
/* ===== HEADER STYLES ===== */
header { background: navy; }
/* Note: font-size set in rem for accessibility */
```

Validate at: <https://jigsaw.w3.org/css-validator/> — checks for syntax errors, invalid properties, and values.

UNIT 4: JAVASCRIPT

Ch. 14 (pg 343–350), Ch. 15 (pg 353–361), Ch. 16 (pg 363–366), Ch. 17 (pg 385–390) | Book [2]

4.1 Introduction to JavaScript

JavaScript (JS) is a lightweight, interpreted programming language that runs directly in the web browser. It makes webpages interactive and dynamic — responding to user actions without reloading the page.

Key Features: Client-side (runs in browser), interpreted (no compilation), event-driven, object-based, loosely typed (variables can hold any type).

Adding JS to HTML: Use `<script>` tags, placed ideally just before `</body>`.

```
<script>
  document.write("Hello, World!")
</script>
```

```
<!-- External JS file -->
<script src="script.js"></script>
```

4.2 Chapter 14: Expressions & Control Flow in JavaScript

Expressions

An expression is a combination of values, variables, operators, and functions that results in a value. Result can be a number, string, or Boolean (true/false).

In JavaScript, these values evaluate to FALSE: false, 0, -0, "" (empty string), null, undefined, NaN. Everything else evaluates to TRUE.

Variables

JavaScript variables are declared using var, let, or const:

```
var name = "Alice";      // old way (function-scoped)
let age = 25;             // modern (block-scoped, can reassign)
const PI = 3.14;         // constant (cannot be reassigned)
```

Data types: Number, String, Boolean, null, undefined, Object, Array.

JS is loosely typed — no need to declare data type; it is determined automatically.

Operators

Operator Type	Operators	Description
---------------	-----------	-------------

Arithmetic	+ - * / % ** ++ --	Basic math. % = modulus (remainder). ** = exponent
Assignment	= += -= *= /= %=	Assign values to variables
Comparison	== != < > <= >=	Compare values; return true or false
Strict Equality	=== !==	Compare value AND type (no type conversion)
Logical	&& !	AND, OR, NOT — combine conditions
String	+	Concatenate (join) strings: "Hi" + " World"
Ternary	? :	Short if-else: condition ? valueIfTrue : valueIfFalse

Operator Precedence (High to Low)

() → ++ -- → * / % → + - - > < >= <= → == != === !== → && → || → ? : → = (assignment) → ,

Equality vs Identity Operators

```
a = 3.14;    b = "3.14"
a == b      // TRUE  - JS converts type, then compares value
a === b     // FALSE - strict: different types (number vs string)
a != b      // FALSE
a !== b     // TRUE
```

The if Statement

```
if (condition) {
  // runs if condition is true
} else if (anotherCondition) {
  // runs if anotherCondition is true
} else {
  // runs if all conditions are false
}
```

Note: JavaScript has NO elseif (one word) — use else if (two words) instead.

The switch Statement

Use when a variable can have multiple values and you want different actions for each:

```
switch (page) {
  case "Home":
    document.write("Home Page")
    break
  case "About":
    document.write("About Us")
    break
  default:
    document.write("Unknown page")
    break
}
```

break: exits the switch block. Without it, execution "falls through" to the next case.

default: runs when no case matches.

The Ternary Operator (? :)

A one-line shorthand for if...else:

```
condition ? expressionIfTrue : expressionIfFalse

// Example:
var result = (a <= 5) ? "Small" : "Large"
document.write(a <= 5 ? "Small" : "Large")
```

Looping Statements

```
// while loop - checks condition BEFORE executing
while (count < 10) {
    document.write(count + "<br>")
    count++
}
```

```
// do...while - executes FIRST, then checks condition
do {
    document.write(count + "<br>")
    count++
} while (count < 10)
```

```
// for loop - most common counting loop
for (var i = 0; i < 5; i++) {
    document.write("Item " + i + "<br>")
}
```

```
// for...in - iterate over object properties
for (var key in myObject) {
    document.write(key + " = " + myObject[key])
}
```

break and continue

break: Immediately exits the current loop or switch statement.

continue: Skips the rest of the current loop iteration and moves to the next.

4.3 Chapter 15: Functions, Objects & Arrays

JavaScript Functions

Functions are reusable blocks of code. Define once, call many times. Functions prevent code repetition and improve organization.

```
// Function definition
function functionName(param1, param2) {
    // statements
    return result;    // optional
```



```
}
```

```
// Function call  
var answer = functionName(value1, value2)
```

Naming convention (camelCase): firstName, getUserData, calculateTotal.
Function names are case-sensitive: getInput ≠ GetInput ≠ GETINPUT.

The arguments Array

Every function has a built-in arguments array — lets a function accept any number of parameters:

```
function displayItems() {  
  for (var j = 0; j < displayItems.arguments.length; j++) {  
    document.write(displayItems.arguments[j] + "<br>")  
  }  
}  
displayItems("Dog", "Cat", "Pony", "Hamster") // works with any number!
```

Returning Values

```
function add(a, b) {  
  return a + b // returns the result to the caller  
}  
var total = add(5, 3) // total = 8
```

A function can also return an array to send back multiple values.

JavaScript Objects

An object groups related data (properties) and functions (methods) together.

```
// Define a class using a constructor function  
function User(forename, username, password) {  
  this.forename = forename  
  this.username = username  
  this.password = password  
  this.showUser = function() {  
    document.write("Name: " + this.forename)  
  }  
}
```

```
// Create an object (instance) of the class  
var user1 = new User("Alice", "alice01", "pass123")
```

```
// Access properties and methods  
document.write(user1.forename) // Alice  
user1.showUser() // Name: Alice
```

```
// Add a new property at any time  
user1.email = "alice@example.com"
```

The prototype Keyword

prototype saves memory: instead of each object having its own copy of a method, all objects share ONE reference to it.

```
User.prototype.showUser = function() {  
    document.write("Name: " + this.forename)  
}  
// All User objects, even already created, inherit this method
```

JavaScript Arrays

Arrays store multiple values in a single variable. Index starts at 0.

```
// Create an array  
var fruits = []           // empty array  
var colors = new Array()  // another way  
var nums    = [1, 2, 3, 4] // array literal
```

```
// Add items  
fruits.push("Apple")      // add to end  
fruits.push("Mango")  
fruits[0] = "Banana"      // set specific index
```

```
// Access items  
document.write(fruits[0]) // Apple  
document.write(fruits.length) // total number of elements
```

Method	Description
push(item)	Add item to the END of the array
pop()	Remove and return the LAST item
shift()	Remove and return the FIRST item
unshift(item)	Add item to the BEGINNING of the array
splice(start, n)	Remove n items starting at index start
slice(start, end)	Return a portion of the array (non-destructive)
sort()	Sort the array alphabetically (in-place)
reverse()	Reverse the order of elements (in-place)
join(separator)	Convert array to string, elements separated by separator
indexOf(item)	Return the index of item (-1 if not found)
concat(arr2)	Combine two arrays into a new array
length	Property: number of elements in the array

4.4 Chapter 16: JavaScript & Validation

JavaScript is commonly used to validate form input on the client side — providing instant feedback without a server round-trip.

Accessing Form Elements

```
// Get element by ID
var name = document.getElementById("username").value
var email = document.getElementById("email").value
```

```
// Get element by name
var form = document.forms["myForm"]
var input = form.elements["email"].value
```

Basic Form Validation Example

```
function validateForm() {
    var name = document.getElementById("name").value
    var email = document.getElementById("email").value

    if (name == "") {
        alert("Name is required!")
        return false // prevent form submission
    }
    if (email.indexOf("@") == -1) {
        alert("Please enter a valid email address!")
        return false
    }
    return true // allow submission
}
```

```
<form onsubmit="return validateForm()">
  <input type="text" id="name" placeholder="Your name">
  <input type="email" id="email" placeholder="Your email">
  <button type="submit">Submit</button>
</form>
```

Regular Expressions in Validation

Regular expressions (RegExp) are patterns used to validate string formats.

```
// Test if email is valid
var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/
if (!emailPattern.test(email)) {
    alert("Invalid email format!")
}
```

```
// Test if input contains only letters
var namePattern = /^[a-zA-Z ]+$/
if (!namePattern.test(name)) {
    alert("Name can only contain letters!")
}
```

String Methods Useful for Validation

Method	Description	Example
--------	-------------	---------

length	Returns string length	"Hello".length → 5
indexOf(str)	Position of str (-1 if not found)	"hello@x".indexOf("@") → 5
toUpperCase()	Convert to uppercase	"hello".toUpperCase() → "HELLO"
toLowerCase()	Convert to lowercase	"HELLO".toLowerCase() → "hello"
trim()	Remove leading/trailing spaces	" hi ".trim() → "hi"
charAt(n)	Character at position n	"Hello".charAt(0) → "H"
substr(start, len)	Substring from start, length len	"Hello".substr(1,3) → "ell"
replace(pat, rep)	Replace pattern with replacement	"Hello".replace("l","L") → "HeLlo"

4.5 Chapter 17: Asynchronous Communication (Ajax)

Ajax (Asynchronous JavaScript and XML) allows webpages to communicate with a server and update content WITHOUT reloading the entire page. This enables dynamic, responsive web applications.

Asynchronous: The browser does NOT wait for the server response — it continues running other code. When the response arrives, a callback function handles it.

XMLHttpRequest Object

The XMLHttpRequest (XHR) object is used to communicate with the server:

```
// Create the object
var request = new XMLHttpRequest()
```

Property/Method	Description
onreadystatechange	Event handler called every time readyState changes
readyState	0=UNSENT, 1=OPENED, 2=HEADERS_RECEIVED, 3=LOADING, 4=DONE
status	HTTP status code: 200=OK, 404=Not Found, 500=Server Error
responseText	Server response as plain text string
responseXML	Server response as XML document
open(method, url, async)	Configure the request (GET/POST, URL, true for async)
send(body)	Send the request (body=null for GET)
setRequestHeader(h, v)	Set a request header (needed for POST with form data)

Making an Ajax Request

```
var request = new XMLHttpRequest()
request.open("GET", "data.php?name=Alice", true)
request.onreadystatechange = function() {
  if (request.readyState == 4 && request.status == 200) {
    // Request complete and successful
    document.getElementById("result").innerHTML = request.responseText
  }
}
```

```

    }
}
request.send(null)

```

JavaScript Event Handlers (from Book [3])

Event handlers connect user actions (events) to JavaScript functions. They can be added inline in HTML or via JavaScript.

Inline event handler:

```



```

Via JavaScript:

```

document.getElementById("myBtn").onclick = function() {
    alert("Button clicked!")
}

```

Event Handler	Triggered When
onblur	An element LOSES focus (user clicks away)
onclick	An element is clicked
ondblclick	An element is double-clicked
onfocus	An element GAINS focus (user clicks in)
onkeypress	A key is pressed (excludes Shift/Alt/Ctrl/Esc)
onkeydown	A key is being pressed (includes ALL keys)
onkeyup	A key is released
onmouseout	Mouse leaves the element
onmouseover	Mouse enters/passes over the element
onmousemove	Mouse moves over the element
onselect	Text inside an input is selected
onsubmit	A form is submitted (great for validation!)
onload	The page/image has fully loaded
onchange	Value of a form element changes
onreset	Form is reset
onresize	Browser window is resized
onscroll	User scrolls the document

```

// Event handlers in action
<input type="text" id="fname"
onfocus="this.style.background='yellow'"
onblur="this.style.background='white'">

```

```

<form onsubmit="return validate()">
...

```

```
</form>
```

UNIT 5: jQuery AND JSON

Ch. 21 (pg 512–526, 530–538 before animation, 542–544) | Book [2] & Ch. 8 (pg 376–377) | Book [4]

5.1 Introduction to jQuery

jQuery is a fast, lightweight JavaScript library that simplifies HTML manipulation, event handling, CSS animation, and Ajax. It is the most widely used JS library — installed on over 73% of all websites.

Why jQuery?: Write less, do more. Cross-browser compatible code, simplified DOM manipulation, easy event handling, built-in animation effects, and a huge plugin ecosystem.

Including jQuery in Your Webpage

Option 1: Download from jquery.com and host it locally:

```
<script src="jquery-3.6.0.min.js"></script>
```

Option 2: Use a CDN (Content Delivery Network) — no download needed:

```
<script  
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>  
<!-- OR Microsoft CDN: -->  
<script src="https://ajax.aspnetcdn.com/ajax/jQuery/jquery-  
3.2.1.min.js"></script>
```

jQuery versions: 1.x (supports older IE), 2.x (dropped IE 6-8), 3.x (modern browsers). Use 3.x for new projects.

5.2 jQuery Syntax & Selectors

jQuery's core syntax: select element(s) → perform action on them.

```
$(selector).action()
```

The \$ symbol is jQuery's alias. Everything starts with \$(). The Document Ready function ensures DOM is loaded before code runs:

```
$(document).ready(function() {  
    // Your jQuery code here — DOM is fully loaded  
})  
// Shorthand:  
$(function() {  
    // Your jQuery code here  
})
```

jQuery Selectors

Selector	jQuery Syntax	Description
Element	\$("p")	All <p> elements

ID	<code>\$("#myId")</code>	Element with id="myId"
Class	<code>\$(".myClass")</code>	All elements with class="myClass"
Universal	<code>\$("*")</code>	All elements
Multiple	<code>\$("h1, h2, p")</code>	Multiple element types
First child	<code>\$("li:first")</code>	First element
Last child	<code>\$("li:last")</code>	Last element
Even rows	<code>\$("tr:even")</code>	All even <tr> elements (0-indexed)
Attribute	<code>\$("[href]")</code>	All elements with href attribute
this	<code>\$ (this)</code>	Current element (inside event handler)

5.3 jQuery DOM Manipulation

Getting and Setting Content

```
// Get / Set inner HTML (including tags)
$("#box").html()           // get HTML content
$("#box").html("<b>New</b>") // set HTML content
```

```
// Get / Set text content (no tags)
$("#box").text()           // get text
$("#box").text("Hello!")   // set text
```

```
// Get / Set form input values
$("#myInput").val()         // get value
$("#myInput").val("default") // set value
```

Getting and Setting Attributes & CSS

```
// Attributes
$("#img").attr("src")           // get src attribute
$("#img").attr("src", "new.jpg") // set src attribute
$("#a").attr("href", "https://x.com") // set href
```

```
// CSS properties
$("#box").css("color")           // get color
$("#box").css("color", "red")     // set one property
$("#box").css({ "color": "red", "font-size": "18px" }) // set multiple
```

Adding & Removing Elements

```
$("#ul").append("<li>New item at end</li>") // add inside, at end
$("#ul").prepend("<li>New item at start</li>") // add inside, at start
$("#p").after("<p>After this paragraph</p>") // add after element
$("#p").before("<p>Before this paragraph</p>") // add before element
$("#box").remove() // remove element and its children
$("#box").empty() // remove only children, keep element
```

5.4 jQuery Event Handling

jQuery makes event handling simple and cross-browser compatible:

```
// Basic event attachment
$("button").click(function() {
    alert("Button was clicked!")
})
```

```
// .on() method – recommended modern approach
$("button").on("click", function() {
    $(this).text("Clicked!")
})
```

```
// Multiple events
$("input").on({
    focus: function() { $(this).css("background", "yellow") },
    blur:  function() { $(this).css("background", "white") }
})
```

jQuery Event Method	Equivalent JS Event	Triggered When
.click(fn)	onclick	Element is clicked
.dblclick(fn)	ondblclick	Element is double-clicked
.blur(fn)	onblur	Element loses focus
.focus(fn)	onfocus	Element gains focus
.keypress(fn)	onkeypress	Key is pressed (printable chars)
.keydown(fn)	onkeydown	Any key is pressed down
.keyup(fn)	onkeyup	Key is released
.mouseover(fn)	onmouseover	Mouse enters the element
.mouseout(fn)	onmouseout	Mouse leaves the element
.mouseenter(fn)	(jQuery only)	Mouse enters (no bubbling)
.mouseleave(fn)	(jQuery only)	Mouse leaves (no bubbling)
.hover(in, out)	(jQuery only)	Combines mouseenter + mouseleave
.submit(fn)	onsubmit	Form is submitted
.change(fn)	onchange	Form element value changes
.on(event, fn)	addEventListener	Attach any event handler

Event Examples

```
// blur and focus events
$("input").focus(function() { $(this).css("background", "yellow") })
$("input").blur(function() { $(this).css("background", "#aaa") })
```

```
// click and dblclick
$("button").click(function() { $(this).slideUp() })
```



```
$("#button").dblclick(function() { $(this).hide() })
```

```
// Form validation on submit
$("#form").submit(function() {
    if ($("#fname").val() == "" || ($("#lname").val() == "") {
        alert("Please fill all required fields!")
        return false // prevent submission
    }
    return true
})
```

```
// Key events
$("#input").keypress(function(event) {
    $("#result").html("Key pressed: " + event.key)
})
```

5.5 jQuery Effects & Animation

jQuery has built-in methods for common visual effects. All effects can accept a speed parameter: "fast", "slow", or milliseconds (e.g., 1000 for 1 second).

Show / Hide

```
$("#element").hide() // instantly hide
$("#element").hide(1000) // hide over 1 second
$("#element").hide("slow") // hide slowly
$("#element").show() // show element
$("#element").show("fast") // show fast
$("#element").toggle() // toggle between show/hide
$("#element").toggle("slow", "linear") // with speed and easing
```

Fade Effects

```
$("#element").fadeIn("slow") // fade in from invisible
$("#element").fadeOut("slow") // fade out to invisible
$("#element").fadeToggle("slow") // toggle fade in/out
$("#element").fadeTo("slow", 0.5) // fade to 50% opacity
```

Slide Effects

```
$("#element").slideUp("slow") // slide up (hide)
$("#element").slideDown("slow") // slide down (show)
$("#element").slideToggle("slow") // toggle slide up/down
```

Chaining Multiple Effects

jQuery allows method chaining — run multiple effects in sequence:

```
$("#element").hide(1000).show(1000)
$("#div").fadeOut("slow").fadeIn("slow")
$("#p").slideUp(500).slideDown(500).css("color", "red")
```

Easing

Easing controls the speed curve of an animation:

swing (default): Starts slow, speeds up, then slows at the end.

linear: Constant speed throughout.

```
$("#box").hide(1000, "linear")    // linear easing
$("#box").show(1000, "swing")    // swing easing (default)
```

5.6 jQuery & CSS

Accessing CSS Properties

```
// Get CSS value
var color = $("#box").css("color")
var width = $("#box").css("width")
```

```
// Set one CSS property
$("#box").css("background-color", "yellow")
```

```
// Set multiple CSS properties at once
$("#box").css({
  "color": "white",
  "background-color": "#1a5276",
  "font-size": "20px",
  "padding": "10px"
})
```

```
// Add / remove / toggle CSS classes
$("#box").addClass("highlight")
$("#box").removeClass("highlight")
$("#box").toggleClass("active")
```

5.7 JSON (JavaScript Object Notation)

JSON is a lightweight data-interchange format. It is human-readable and easy for machines to parse. JSON is the standard format for exchanging data between a web server and client (via Ajax).

Key Rule: JSON keys MUST be strings (in double quotes). Values can be: string, number, boolean, null, array, or object.

JSON Syntax

```
{
  "name": "Alice",
  "age": 25,
  "active": true,
  "scores": [85, 90, 78],
  "address": {
    "city": "Delhi",
    "pin": "110001"
  }
}
```

```
}  
}
```

JSON vs JavaScript Object

Feature	JSON	JavaScript Object
Keys	Must be in "double quotes"	Can be unquoted identifiers
Strings	Must use "double quotes"	Can use single or double quotes
Functions	NOT allowed	Allowed as property values
Trailing commas	NOT allowed	Allowed in modern JS
Use	Data transport/storage format	Runtime data structure

Parsing JSON in JavaScript

```
// JSON string received from server  
var jsonString = '{"name":"Alice","age":25}'
```

```
// Parse JSON string → JavaScript object  
var obj = JSON.parse(jsonString)  
document.write(obj.name)    // Alice  
document.write(obj.age)     // 25
```

```
// Convert JavaScript object → JSON string  
var person = { name: "Bob", age: 30 }  
var json = JSON.stringify(person)  
document.write(json)        // {"name":"Bob","age":30}
```

jQuery \$.getJSON() — Fetch JSON via Ajax

```
$.getJSON("data.json", function(data) {  
    // data is already parsed as a JS object  
    $("#result").text(data.name)  
    $.each(data.items, function(index, value) {  
        $("ul").append("<li>" + value + "</li>")  
    })  
})
```

jQuery \$.ajax() — Full Control Ajax

```
$.ajax({  
    url: "server.php",           // server endpoint  
    type: "GET",                 // or "POST"  
    dataType: "json",           // expect JSON response  
    data: { id: 5 },             // data to send  
    success: function(data) {  
        console.log(data)        // handle success  
        $("#output").html(data.message)  
    },  
    error: function(xhr, status, error) {  
        console.log("Error: " + error) // handle error  
    }  
})
```

```
}
})
```

Shorthand Ajax Methods

Method	Description
\$.get(url, data, callback)	Shorthand GET request
\$.post(url, data, callback)	Shorthand POST request
\$.getJSON(url, data, callback)	GET request, automatically parses JSON response
\$.load(url)	Load HTML from server into selected element

```
// $.get example
$.get("server.php", { name: "Alice" }, function(data) {
    $("#result").html(data)
})
```

```
// $.post example
$.post("login.php", { user: "alice", pass: "123" }, function(data) {
    if (data == "ok") alert("Login successful!")
})
```

5.8 jQuery Selectors in Detail (Summary)

Category	Selector	Example
Basic	\$("element"), \$("#id"), \$(".class")	\$("p"), \$("#nav"), \$(".btn")
Hierarchy	\$("parent child"), \$("parent > child")	\$("ul li"), \$("div > p")
Filter	\$("el:first"), \$("el:last"), \$("el:even"), \$("el:odd")	\$("li:first"), \$("tr:even")
Attribute	\$("[attr]"), \$("[attr=val]")	\$("[href]"), \$("[type='text']")
Content	\$("el:contains(text)")	\$("p:contains('hello')")
Form	\$(":input"), \$(":text"), \$(":checkbox")	\$(":submit"), \$(":checked")

QUICK REFERENCE SUMMARY

CSS Quick Reference

Category	Key Properties
Types	Inline > Embedded > External (cascade order)
Selectors	element, #id, .class, *, a:hover, a b, a,b
Box Model	content → padding → border → margin
Text	font-family, font-size, color, text-align, font-weight
Layout	display, float, clear, position, flex, grid
Background	background-color, background-image, background-size

JavaScript Quick Reference

Topic	Key Syntax
Variables	var x; let x; const X
Conditions	if...else if...else, switch...case, condition ? a : b
Loops	while, do...while, for(init;cond;incr), for...in
Functions	function name(params) { return val; }
Objects	new ClassName(), this.prop, obj.method()
Arrays	[], push, pop, sort, splice, length, forEach
DOM	document.getElementById, innerHTML, style
Events	onclick, onblur, onfocus, onsubmit, onmouseover

jQuery Quick Reference

Category	Key Methods
Ready	\$(function() { ... })
DOM Get	.html(), .text(), .val(), .attr(), .css()
DOM Set	.html(x), .text(x), .val(x), .css(p,v)
DOM Add	.append(), .prepend(), .after(), .before(), .remove()
Events	.click(), .blur(), .focus(), .keypress(), .submit(), .on()
Show/Hide	.hide(), .show(), .toggle()
Fade	.fadeIn(), .fadeOut(), .fadeToggle(), .fadeTo()
Slide	.slideUp(), .slideDown(), .slideToggle()
Ajax	\$.get(), \$.post(), \$.getJSON(), \$.ajax()
JSON	JSON.parse(str), JSON.stringify(obj)

— End of Notes for Units 3, 4 & 5 —